

TECH TALK 15

크랙의 SSE → WebSocket 전환기

크랙(Crack) 서비스의 실시간 스트리밍 아키텍처 개선

SECTION 01

인트로

01

크랙(Crack) — AI 캐릭터 챗 서비스



SECTION 02

기존 SSE에서 발생한 문제점

02

02 기존 SSE에서 발생한 문제점



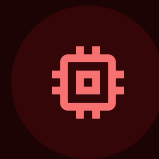
불안정한 연결

네이티브 환경에서
인터넷 연결이 불안정



재연결 어려움

멀티 파드 환경에서
SSE 재연결의 어려움



버퍼링 부담

스트리밍 지속을 위해
공유 스토어에 스트림 버퍼링 필요

→ WebSocket으로 전환 결정

02 기존 SSE에서 발생한 문제점



크랙 BOT 10:25 AM

당신의 AI 서포터, 튀튼입니다.

남겨주신 문의는 운영시간 내 순차적으로 답변드립니다.

- 산업안전보건법에 따라 고객응대근로자를 보호하고 있습니다.
- 폭언시 상담이 제한되며 관련 법령에 따라 처벌 받을 수 있습니다.



데드라인 : 까불면 죽는다 🔒



안다카버 131 | 9월 17일 (금) | 15:28 | 학교 세

안다카버에게 메시지 보내기

8월 09일

안다카버

안다카버

안다카버

안다카버

안다카버

안다카버

공지

고객센터

X

로그아웃

운영 시간이 아닐 때, 안내 메시지 보내기(가) 시작되었습니다. 05:33 PM

02 기존 SSE에서 발생한 문제점



크랙 BOT 10:25 AM

당신의 AI 서포터, 튀튼입니다.

남겨주신 문의는 운영시간 내 순차적으로 답변드립니다.

- 산업안전보건법에 따라 고객응대근로자를 보호하고 있습니다.
- 폭언시 상담이 제한되며 관련 법령에 따라 처벌 받을 수 있습니다.



데드라인 : 까불면 죽는다



답변이 공백으로 나오는 이슈



최하람

답변 비교 3/3



메시지 보내기

* 상황추가



연다카비

#131 | 9월 17일 (금) | 15:28 | 학교 세



연다카비에게 메시지 보내기

05월 09일

연다카비

연다카비

연다카비

연다카비

연다카비

연다카비

연다카비

연다카비

연다카비

연다카비

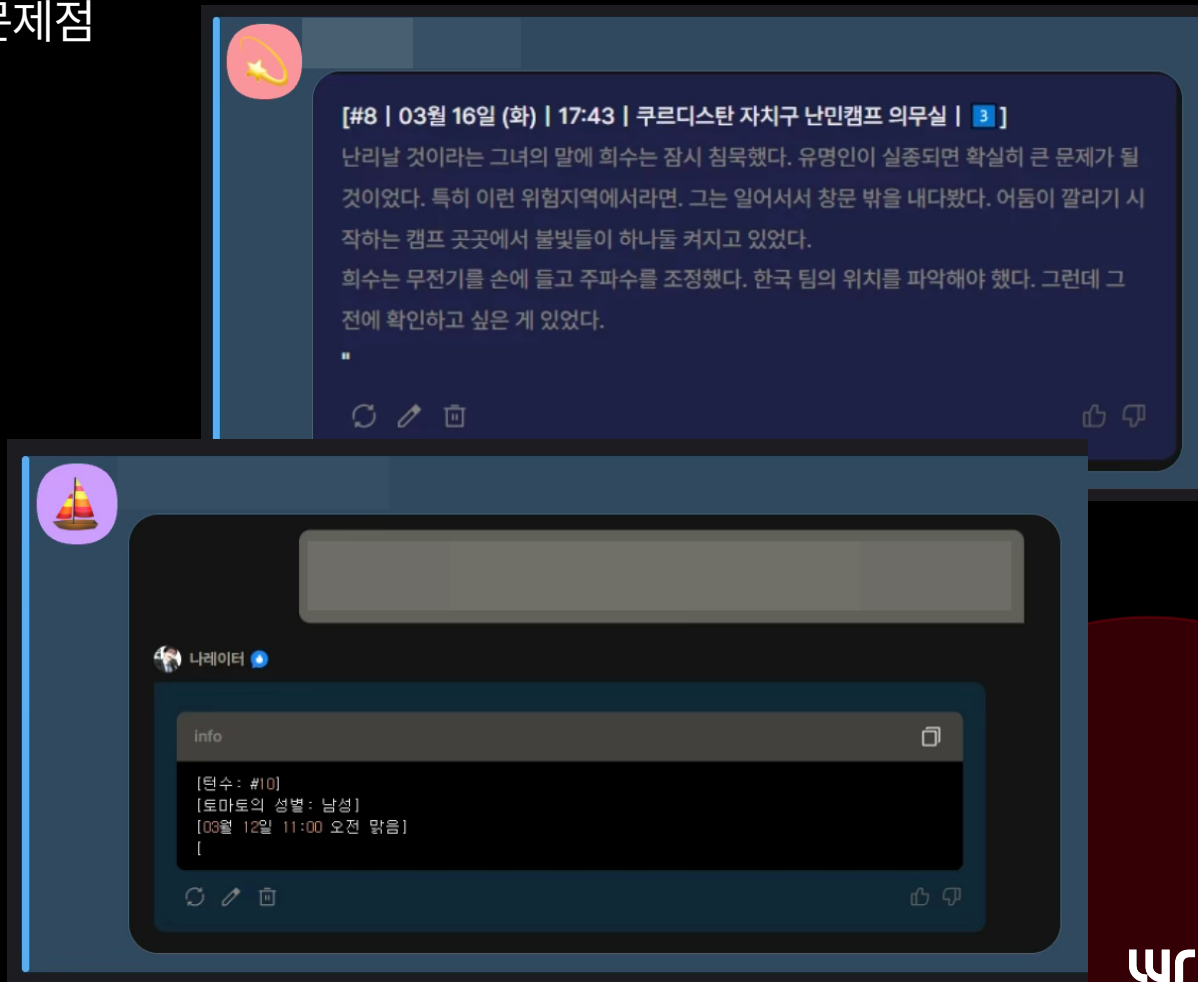
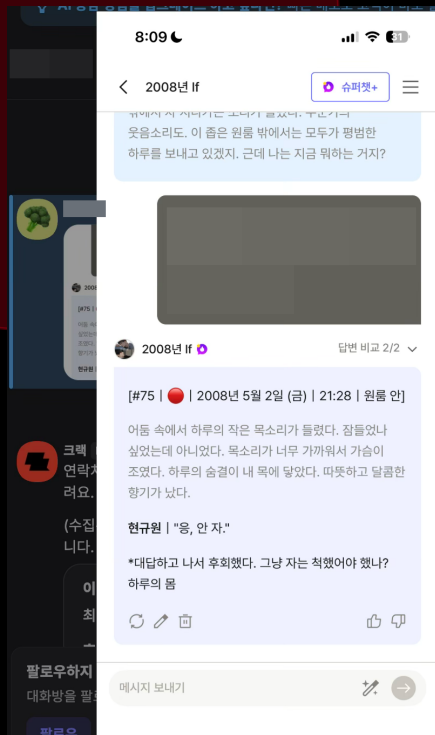
연다카비

연다카비

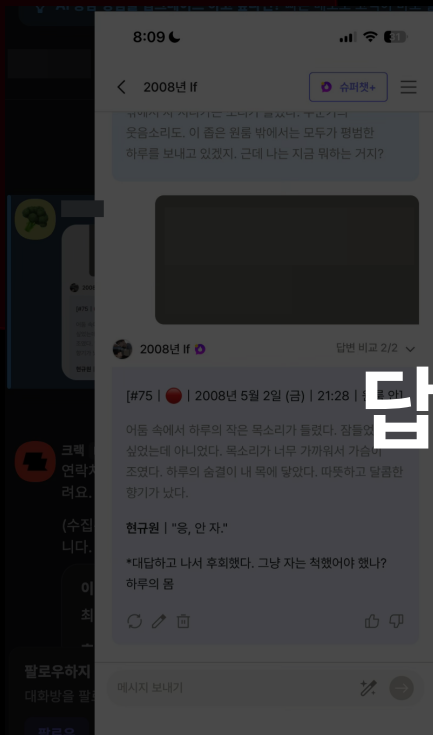
연다카비

운영 시간이 아닐 때, 안내 메시지 보내기(가) 시작되었습니다. 05:33 PM

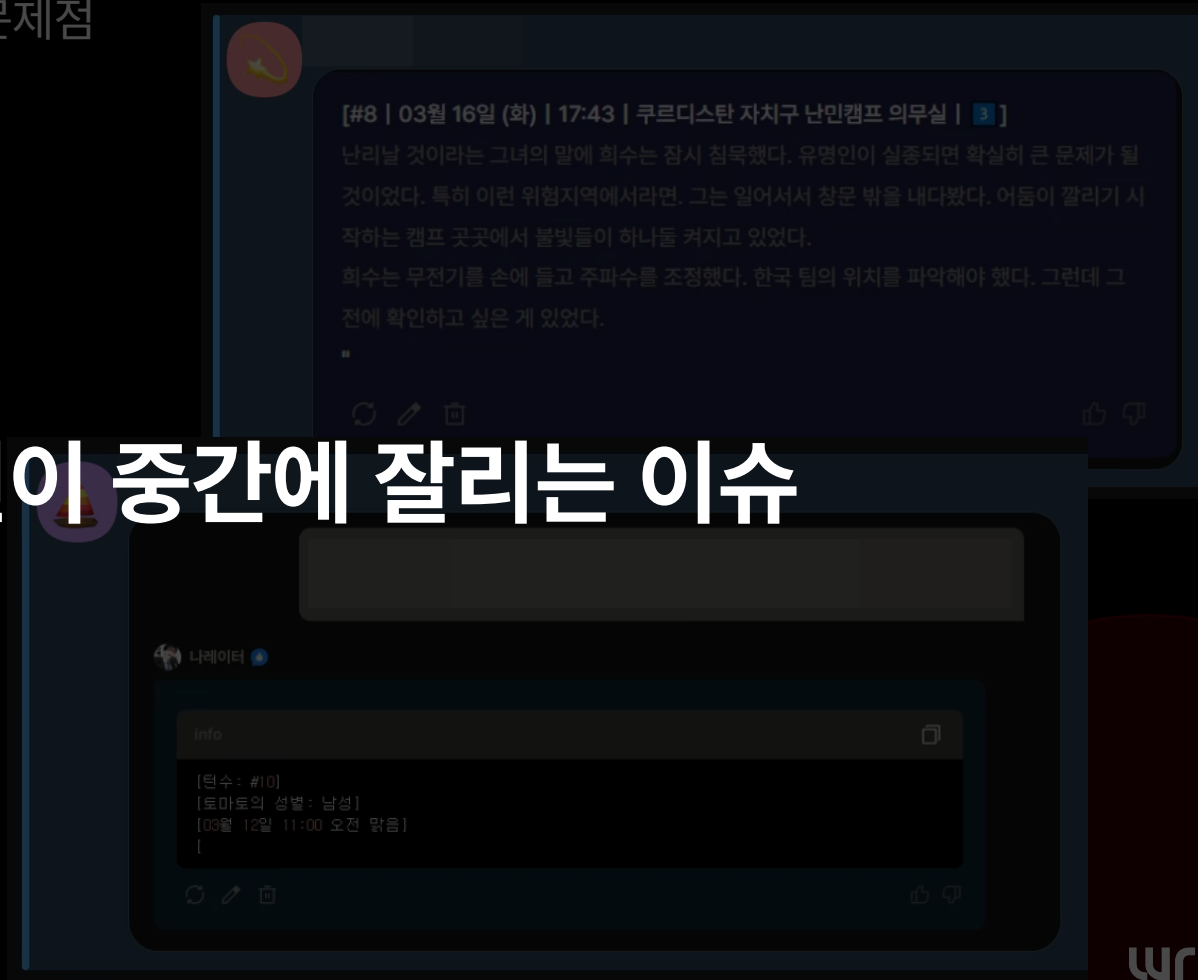
02 기존 SSE에서 발생한 문제점



02 기존 SSE에서 발생한 문제점



답변이 중간에 잘리는 이슈



02 기존 SSE에서 발생한 문제점



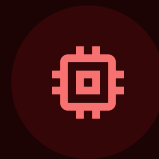
불안정한 연결

네이티브 환경에서
인터넷 연결이 불안정



재연결 어려움

멀티 파드 환경에서
SSE 재연결의 어려움



버퍼링 부담

스트리밍 지속을 위해
공유 스토어에 스트림 버퍼링 필요

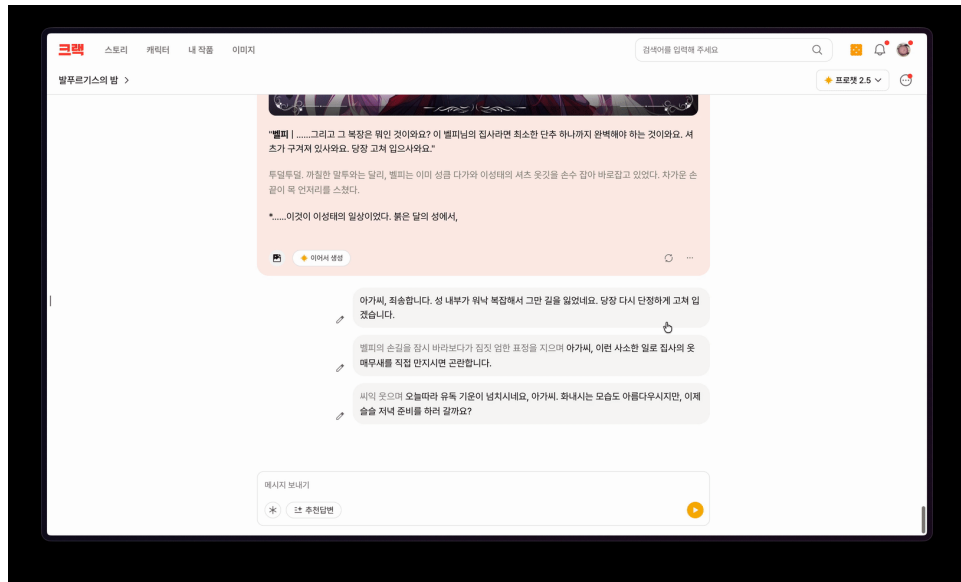
→ WebSocket으로 전환 결정

02 기존 SSE에서

네이티브 환경에서 인터넷 연결이 불안정

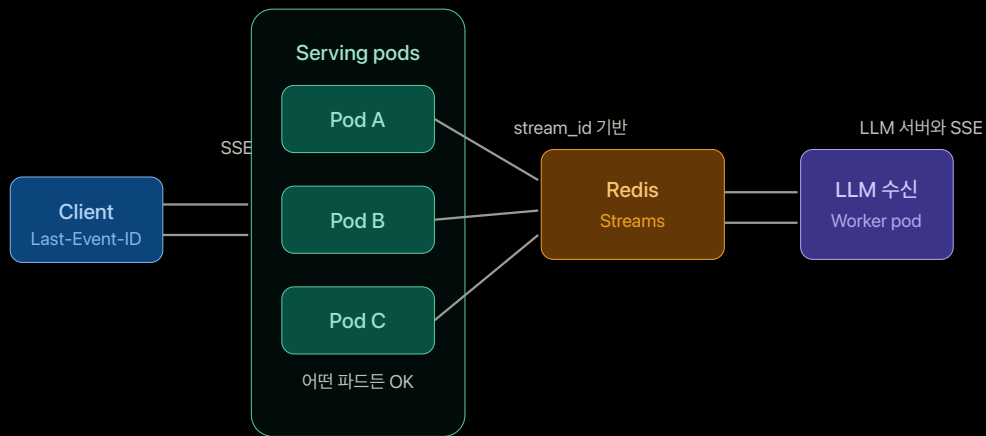
멀티 파드 환경에서 SSE 재연결의 어려움

멀티 파드 환경에서 스트리밍을 지속하려면
기존 세션을 알고 있어야 함



02 기존 SSE에서 발생한 문제점

1. 공유 스토어에 스트림 버퍼링



재연결 시: 아무 파드 → Redis에서 offset 이후 이벤트 재생

02 기존 SSE에서

3. 폴링 하이브리드

```
127     async () => {
128       console.count("redis call");
129       await this.redisService.set(`assistant-content:${message}`);
130     }
131   }
132   );
133
134   readable.on("error", async (err: any) => {
135     console.countReset("redis call");
136     errorObj = handleErrorMessage(res, err);
137     chatLogger.logError(`[claude] ${err.message ?? "stream error"}`);
138   });
```

문제 127 출력 디버그 콘솔 터미널 테스트 결과 포트

터미널

```
redis call: 175
redis call: 176
redis call: 177
redis call: 178
redis call: 179
redis call: 180
redis call: 181
redis call: 182
redis call: 183
redis call: 184
redis call: 185
redis call: 186
redis call: 187
redis call: 188
redis call: 189
redis call: 190
redis call: 191
redis call: 192
```

중단점

- 조사식
- 호출 스택
- 로드된 스크립트
- 중단점
- catch된 예외
- 포착되지 않은 예외
- character-temp-chat-api.service.ts src/api/cha
- character-temp-chat.service.ts src/domain/cha
- character-temp-chat.service.ts src/domain/cha
- character-temp-chat.service.ts src/domain/cha
- character-temp-chat.service.ts src/domain/cha
- character-temp-chat.service.ts src/domain/cha
- chat-api-v3.facade.js dist/api/chat
- chip.repository.ts src/domain/chip
- chip.service.ts src/domain/chip
- claude-chat.service.ts src/api/chat/chat-main
- claude-chat.service.ts src/api/chat/chat-main
- claude-chat.service.ts src/api/chat/chat-main
- claude-chat.service.ts src/api/chat/chat-main
- claude-chat.service.ts src/api/chat/chat-main

02 기존 SSE에서

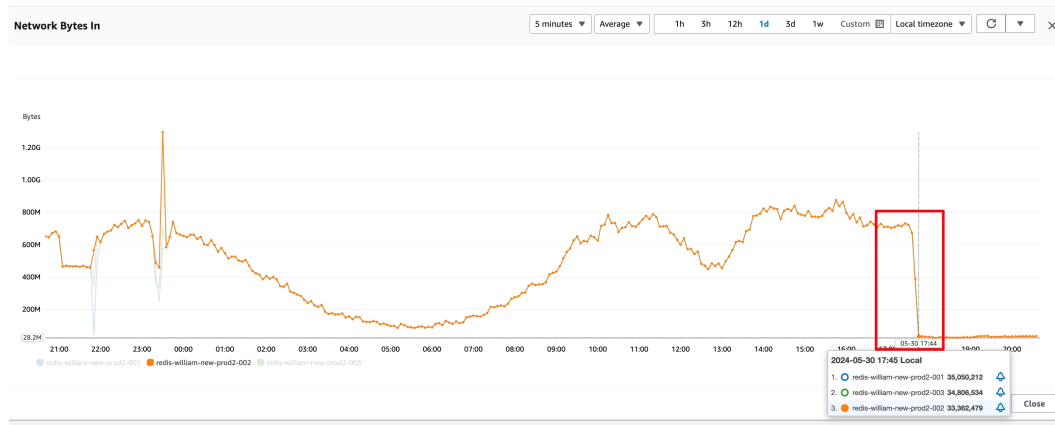
3. 폴링 하이브리드

Resources 24 resources

RESOURCE NAME	↓ REQUESTS	TOTAL TIME	P95 LATENCY	ERRORS	ERROR RATE	MONITORS
☆ SETEX	694M	1.2 y	78.7 ms	13	< 0.1%	
☆ GET	12.9M	1.9 mo	12.4 ms	143	< 0.1%	
☆ LRANGE	10.8M	6.2 mo	14.5 ms	99	< 0.1%	
☆ LSET	2.61M	1.2 d	58.6 ms	0	0%	
☆ LPUSH	2.45M	1.4 d	59.5 ms	0	0%	
☆ EXPIRE	1.31M	19 h 22 min	61.4 ms	0	0%	
☆ MULTI	1.06M	17 h 35 min	59.5 ms	0	0%	
☆ EXEC	1.06M	17 h 32 min	59.5 ms	0	0%	
☆ UNLINK 	287k	1 h 48 min	56.8 ms	0	0%	
☆ HSET	73.2k	1 h 13 min	62.3 ms	0	0%	

02 기존 SSE에서

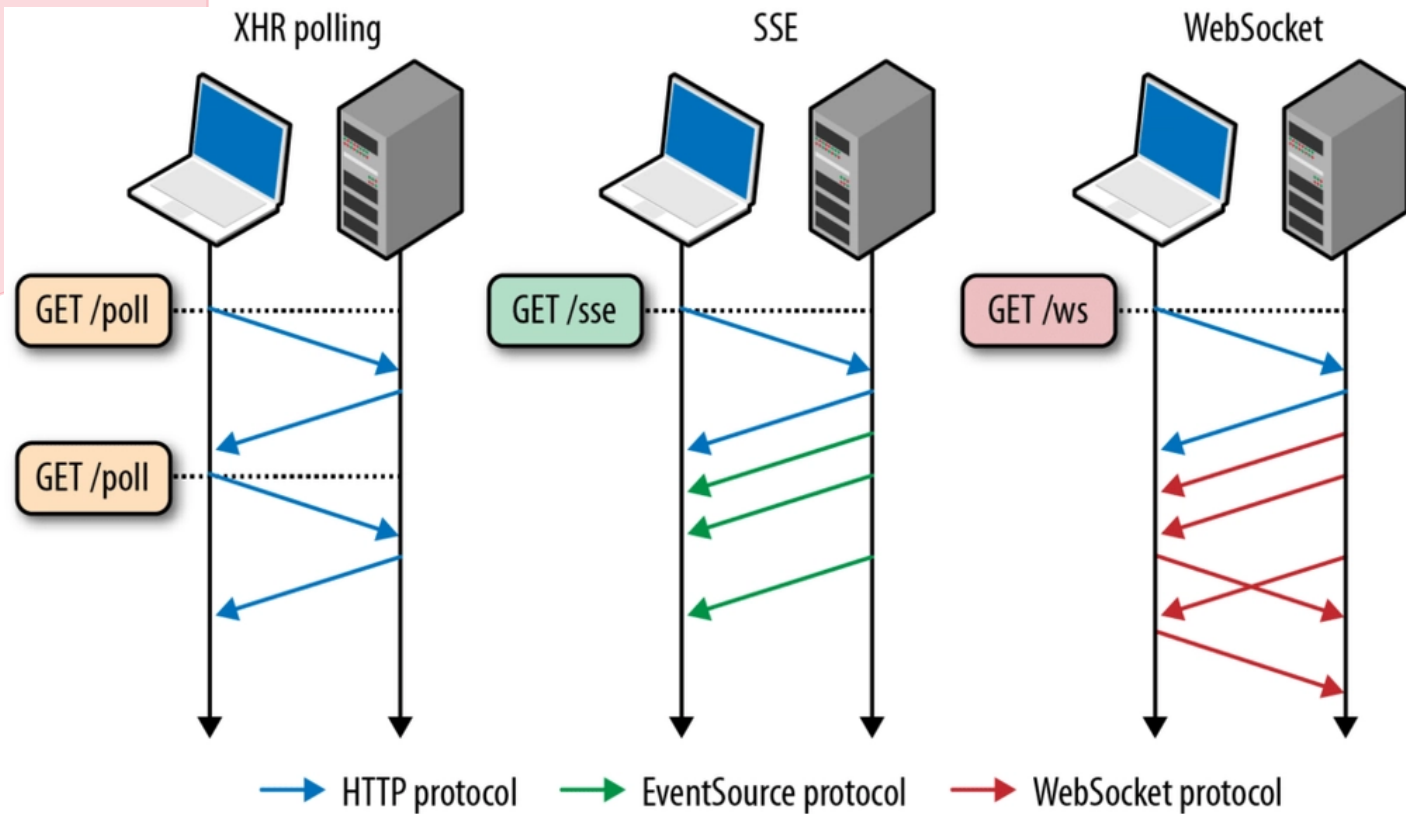
3. 폴링 하이브리드



SECTION 03

웹소켓으로 전환

03



SECTION 04

문제점 1: Redis 대역폭 초과

Redis Network Outbound Bandwidth Exceeded



뤼튼 내 다른 웹소켓 서버에서 사용하는 ElastiCache 인스턴스에서
Network bandwidth limit을 초과하는 문제 발생

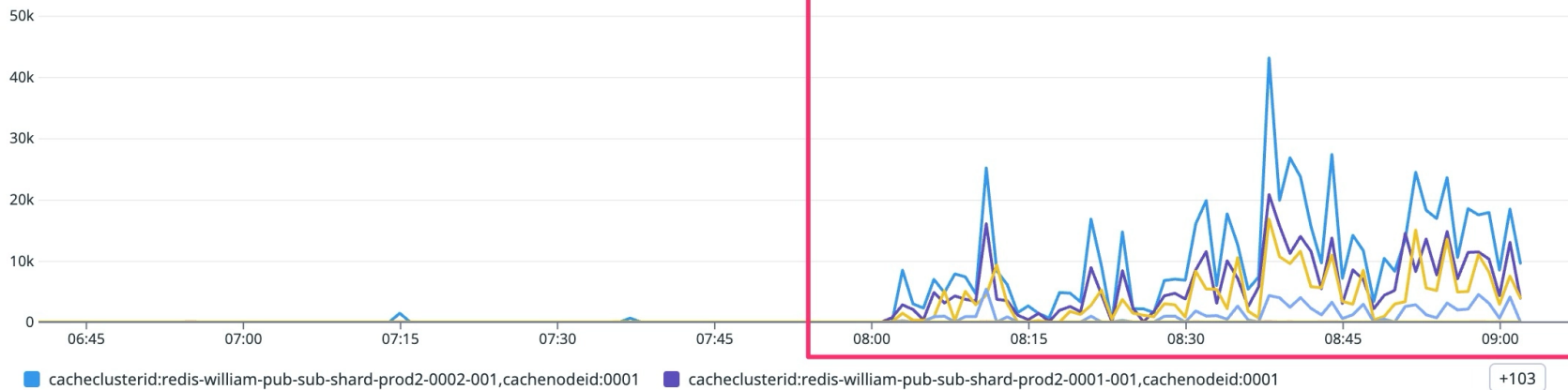
Socket.io Redis Adapter의 구조적 문제

모든 파드가 모든 채널을 구독

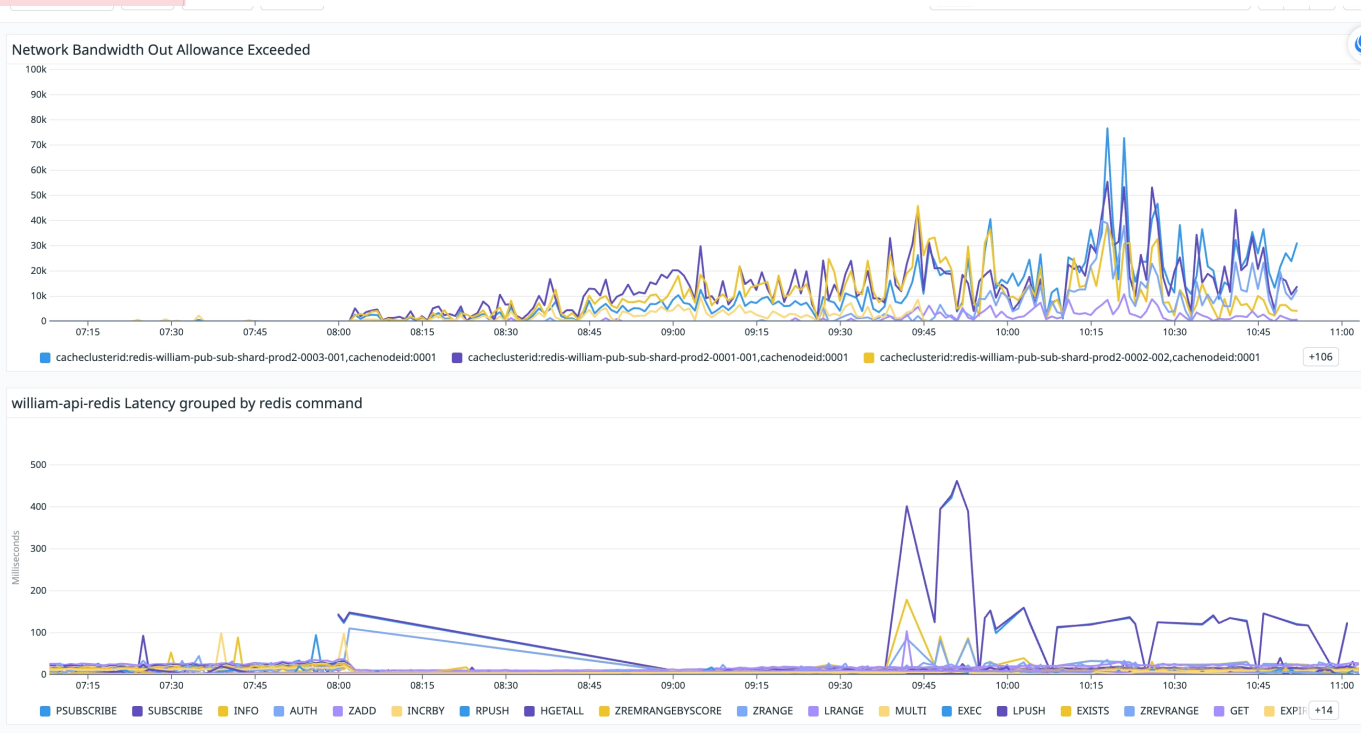
- 스트리밍을 이어나가려고 모든 파드가 특정 유저의 chunk를 수신
- 불필요한 네트워크 트래픽이 기하급수적으로 증가

04 문제점: Redis

Network Bandwidth Out Allowance Exceeded



04 문제점: Redis

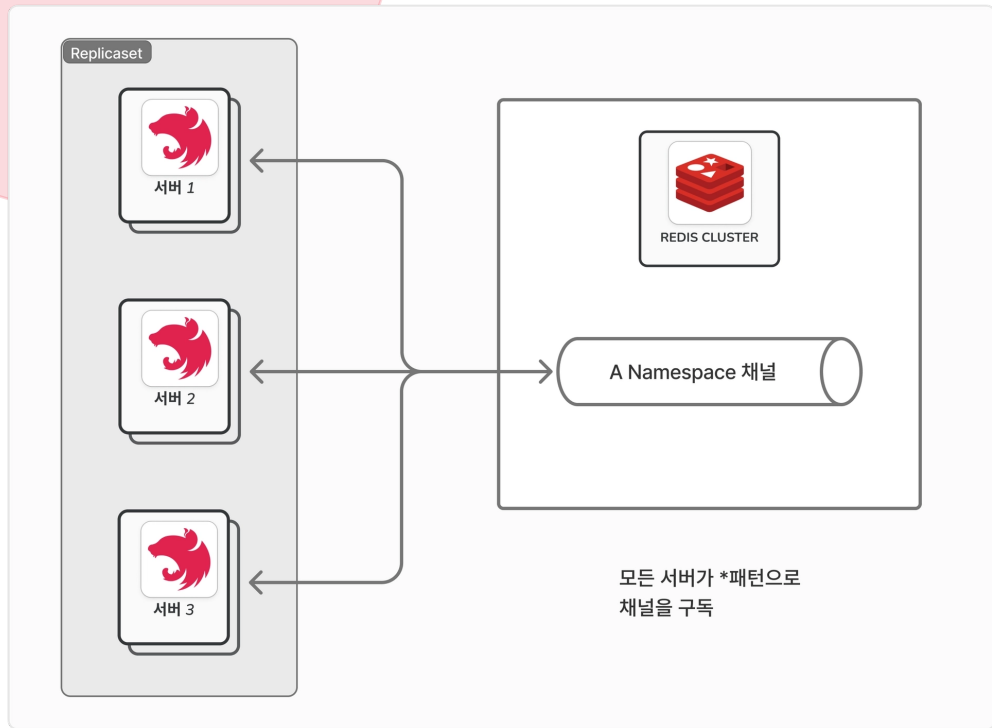


04 문제점: Redis

```
export class RedisAdapter extends Adapter {
  private readonly channel: string;

  constructor(
    nsp: any,
    readonly pubClient: any,
    readonly subClient: any,
    opts: Partial<RedisAdapterOptions> = {}
  ) {
    ...
    this.channel = prefix + "#" + nsp.name + "#"; // 특정 endpoint를 channel로 관리
    ...
    this.subClient.pSubscribe(
      this.channel + "*", // 특정 endpoint 이벤트를 모두 구독
      this.redisListeners.get("psub"),
      true
    );
    ...
  }
}
```

04 문제점: Redis



스트리밍을 이어 나가려고 모든 파드가
특정 유저의 chunk를 수신

세션이 연결되지 않은 파드들도 모두
이벤트를 수신하므로 비효율적인 네트워크
트래픽 사용

04 문제점: Redis

The image displays a development environment with two main windows: Postman and VS Code.

Postman (Left Window): Shows a REST client for a chat service. The collection is named "캐릭터 챗 소켓(Terry)". The endpoint is "((characterChatUrl)) /v3/chats". The request body is a JSON object: `{ "chatId": "6879d3a2436b1d38a893868b" }`. The response is a JSON object: `{ "chatId": "6879d3a2436b1d38a893868b" }`. The status is "Disconnected".

VS Code (Right Window): Shows the source code of a Redis adapter. The file is `node_modules/_index.js`. The code defines a `RedisAdapter` class that extends `socket.io.adapter_1.Adapter`. It implements the `onmessage` method, which handles chat messages by checking if the message is a subscription message or a private message. The code is as follows:

```
class RedisAdapter extends socket.io.adapter_1.Adapter {
  constructor(nsp, publicClient, subClient, opts = {}) {
    this.subClient.on("error", this.friendlyErrorHandler);
  }
  /**
   * Called with a subscription message
   */
  * @private
  onmessage(pattern, channel, msg) {
    channel = channel.toString();
    const channelMatches = channel.startsWith(this.channel);
    if (!channelMatches) {
      return debug("ignore different channel");
    }
    const room = channel.slice(this.channel.length, -1);
    if (room !== "" && !this.hasRoom(room)) {
      return debug("ignore unknown room %s", room);
    }
    const args = this.parses.decode(msg);
    const [uid, packet, opts] = args;
    if (this.uid === uid) {
      return debug("ignore same uid");
    }
    if (packet && packet.nsp === undefined) {
      packet.nsp = "/";
    }
    if (!packet || packet.nsp !== this.nsp.name) {
      return debug("ignore different namespace");
    }
  }
}
```

The bottom of the VS Code window shows a debug console with logs from the application. The logs are as follows:

```
at async NativeConnection.startSession (/Users/iseongtae/Desktop/Programming/contents-server/node_modules/pnpm/mongoose@8.15.1/node_modules/mongoose/lib/connection.js:688:3)
at async MongooseService.getSession (/Users/iseongtae/Desktop/Programming/contents-server/lib/infrastructure/src/providers/mongodb/mongodb.service.ts:11:12)
at async MongooseService.withTransaction (/Users/iseongtae/Desktop/Programming/contents-server/lib/infrastructure/src/providers/mongodb/mongodb.service.ts:19:21)
at async ChatV3WsService.generateCharacterMessage (/Users/iseongtae/Desktop/Programming/contents-server/apps/character-chat/src/application/chat-v3/chat-v3-ws.service.ts:391:24)
```


SECTION 05

해결: Selective Adapter

05 해결: Selecti

```
export class SelectiveIoAdapter extends Adapter {

  constructor(
    nsp: any,
    readonly pubClient: Redis,
    readonly subClient: Redis,
    opts: Partial<RedisAdapterOptions> = {}
  ) {
    // 웹소켓 메시지 처리를 위한 콜백 처리
    this.redisListeners.set("messageBuffer", this.onmessage.bind(this));
    this.subClient.on("messageBuffer", this.redisListeners.get("messageBuffer"));
  }

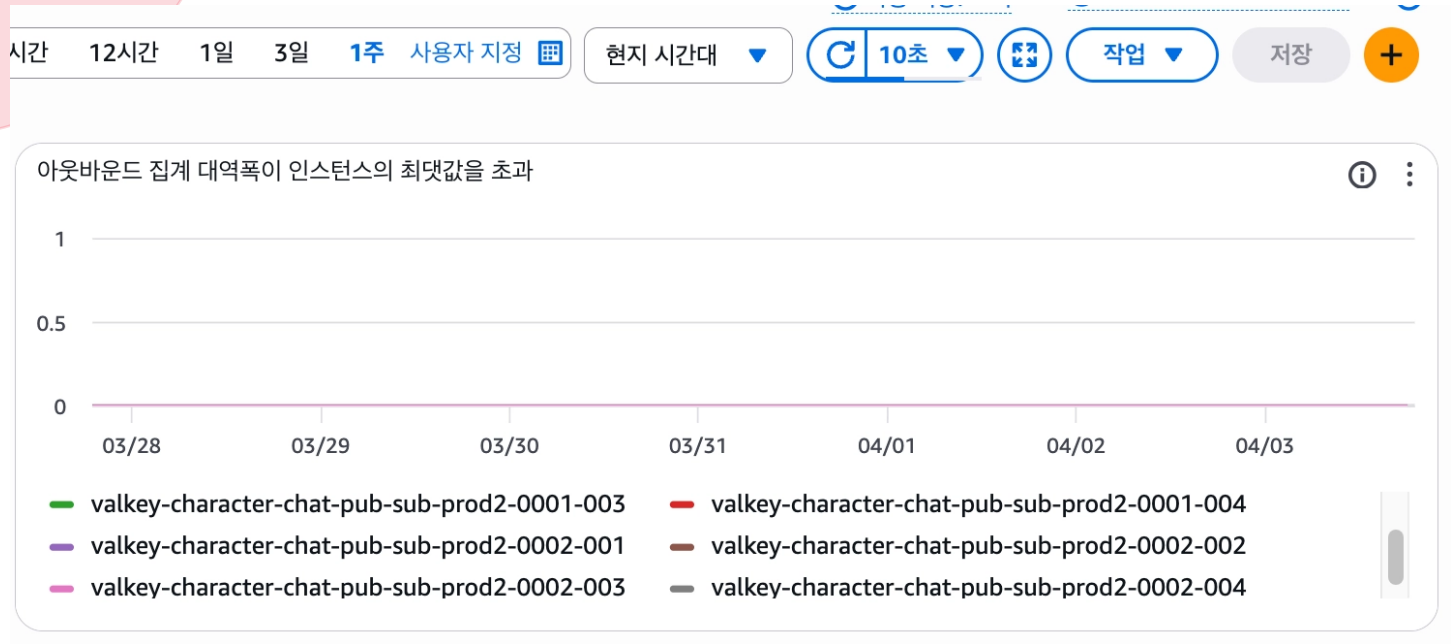
  /**
   * 소켓 커넥션을 맺거나, room에 join하는 경우 호출되는 메서드
   */
  public addAll(id: SocketId, rooms: Set<Room>): Promise<void> | void {
    // ... 생략
    if (isMongoId(room)) {
      // 서버에서 room이 생성되면 그 시점에 특정 Room을 채널로 구독
      this.subClient.subscribe(this.getRoomChannel(room));
    }
    // ... 생략
  }
}
```

SECTION 06

성능 개선 결과

06

06 성능 개



06 성능 개

DB 메모리 사용률



Percent



- valkey-character-chat-pub-sub-dev-0001-001
- valkey-character-chat-pub-sub-dev-0001-002
- valkey-character-chat-pub-sub-dev-0001-003
- valkey-character-chat-nih-sub-dev-0002-001

CPU 사용률



Percent

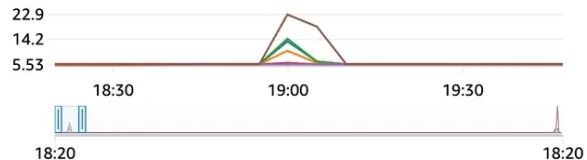


- valkey-character-chat-pub-sub-dev-0001-001
- valkey-character-chat-pub-sub-dev-0001-002
- valkey-character-chat-pub-sub-dev-0001-003
- valkey-character-chat-nih-sub-dev-0002-001

DB 메모리 사용률



Percent

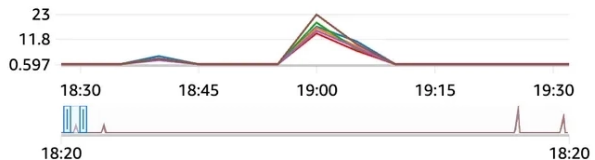


- valkey-character-chat-pub-sub-dev-0001-001
- valkey-character-chat-pub-sub-dev-0001-002
- valkey-character-chat-pub-sub-dev-0001-003
- valkey-character-chat-nih-sub-dev-0002-001

CPU 사용률



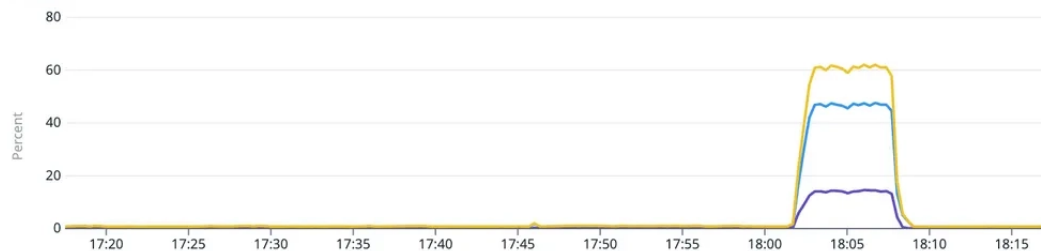
Percent



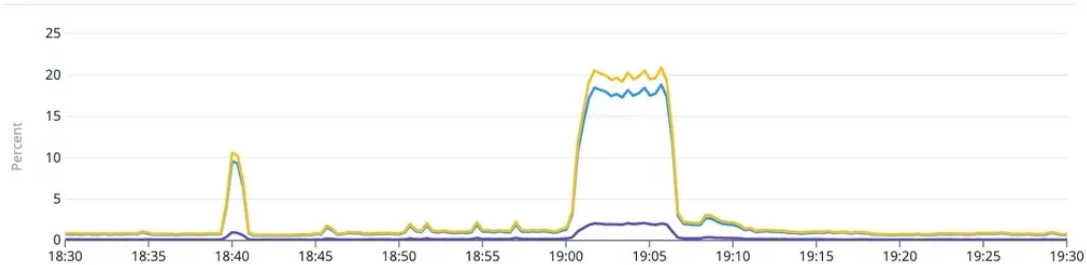
- valkey-character-chat-pub-sub-dev-0001-001
- valkey-character-chat-pub-sub-dev-0001-002
- valkey-character-chat-pub-sub-dev-0001-003
- valkey-character-chat-nih-sub-dev-0002-001

06 성능 개

CPU Usage



CPU Usage

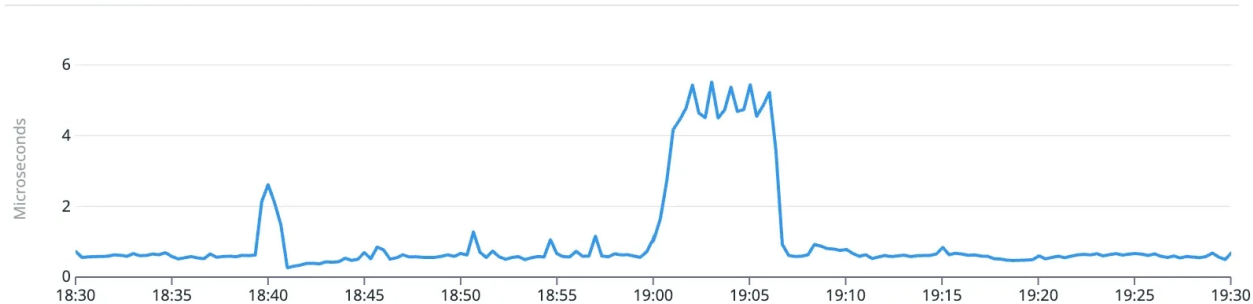


06 성능 개

Garbage Collection Pause Time



Garbage Collection Pause Time

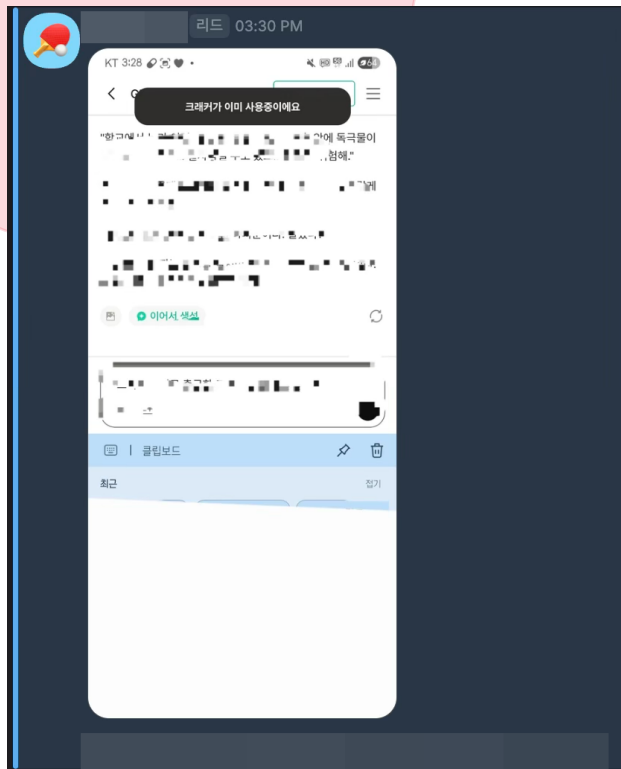


SECTION 07

07

문제점 2: Graceful Shutdown

07 문제점 2: Gra



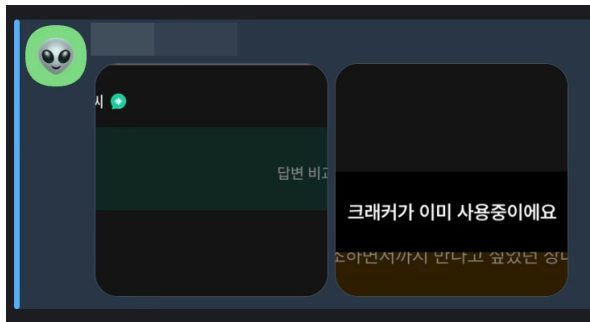
 채팅을 치면 이미 크래커를 사용하고 있다고 하고 채팅이 안쳐져요



채팅 쓰지도 않았는데 [크래커가 사용중이에요]라고만 뜨면서 다음 채팅이 안 보내집니다. 어떤 채팅방에도 보내지지 않습니다.



크래커가 사용중이라며 전체 다른 방도 채팅을 사용할수 없어요ㅠㅠㅠ



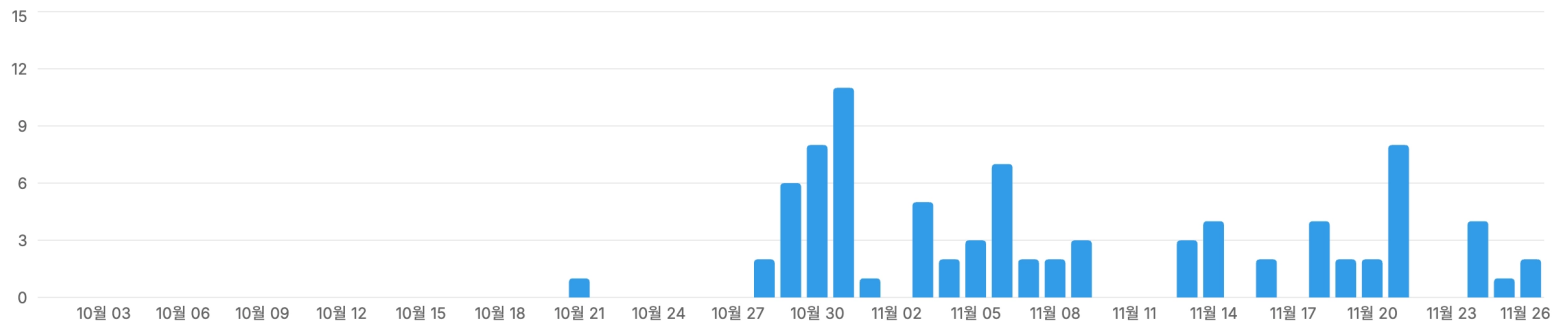
07 문제점 2: Gra

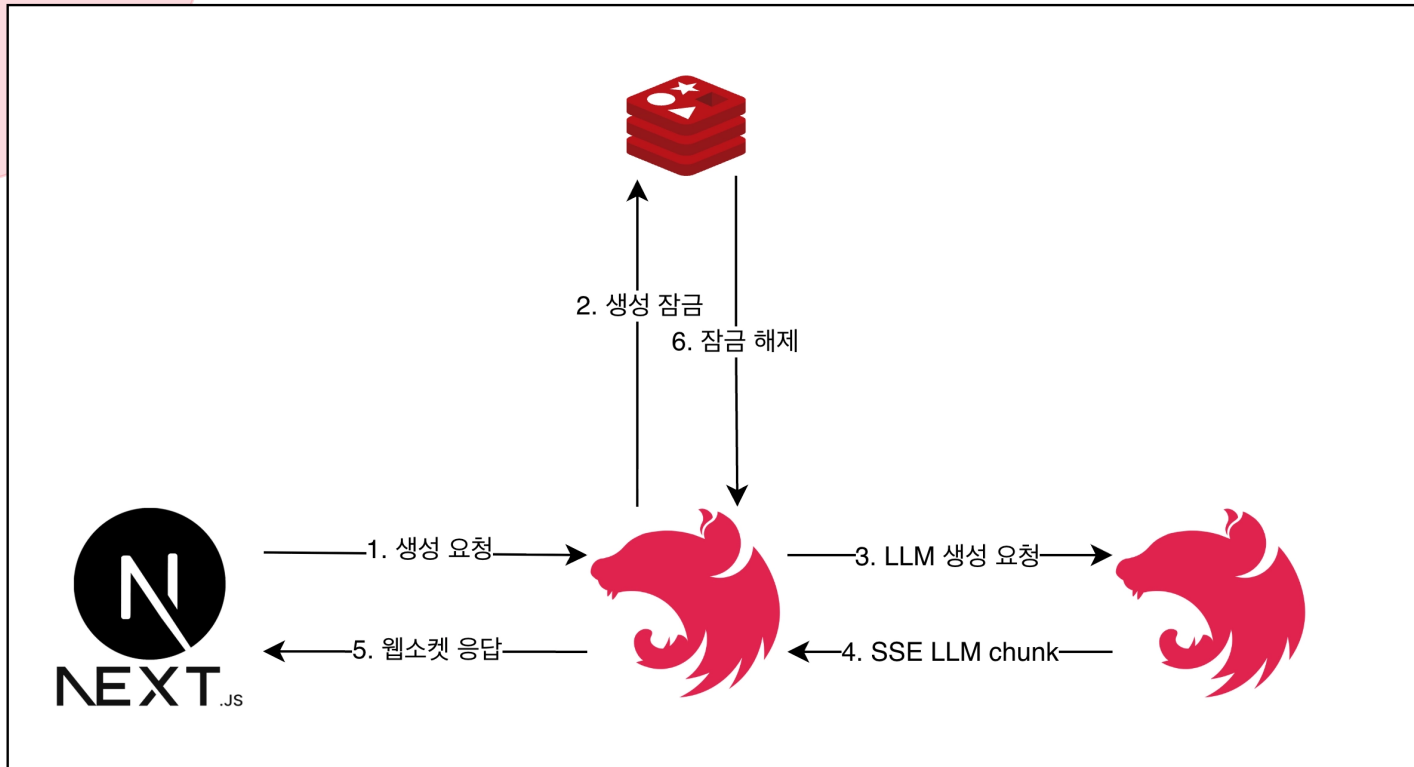
전체 상담 ?

85 ↑ 73

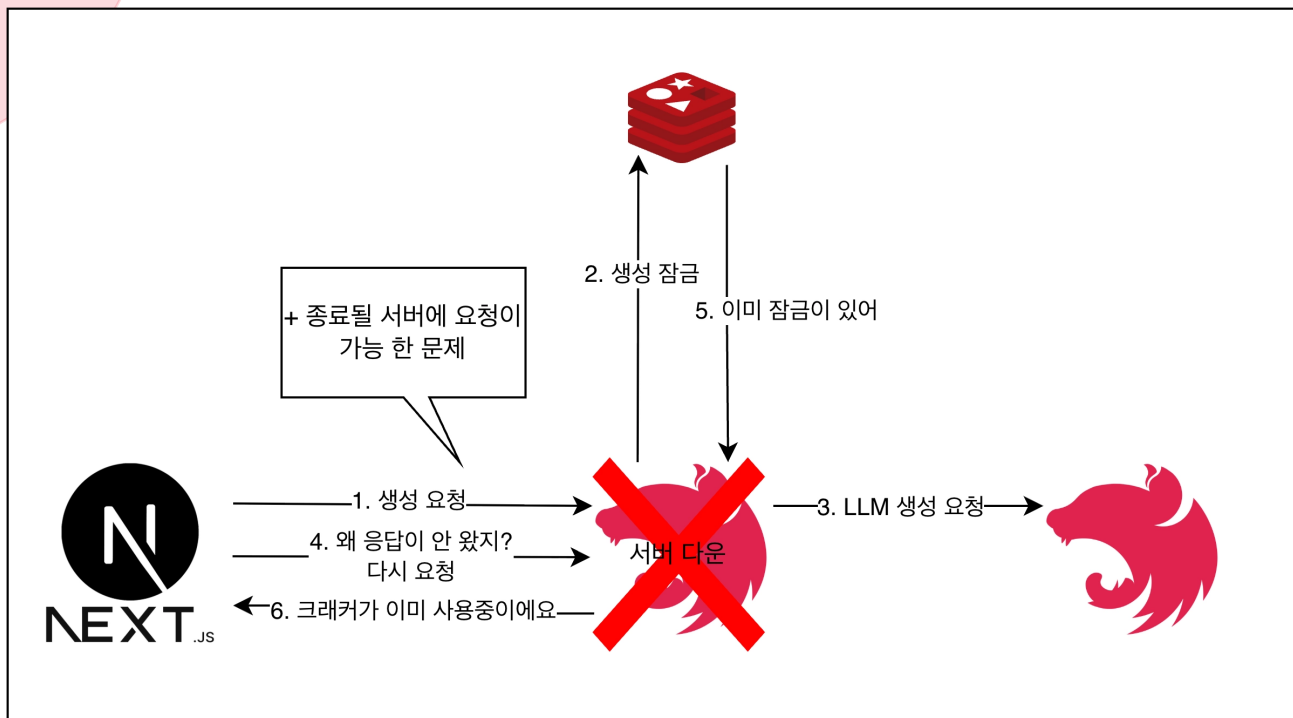
일

주





크래커가 이미 사용중이에요 오류 케이스

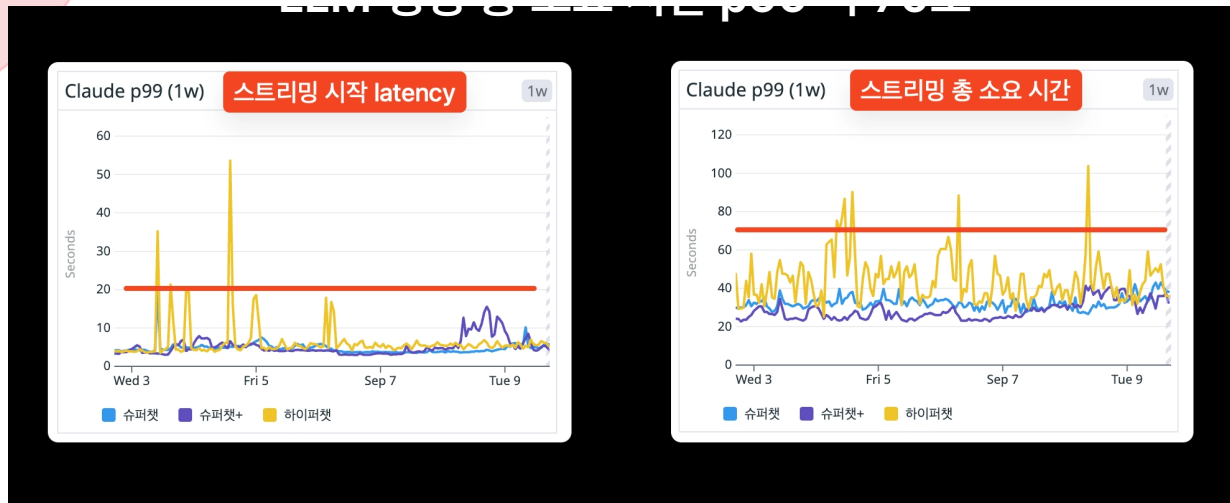


SECTION 08

해결: Graceful Shutdown 적용

08 해결 방안 - terminationGracePeriodSeconds 조정

롤링 업데이트



스트리밍 총 소요 시간보다 더 길게 terminationGracePeriodSeconds을 조정

08 해결 방안 - 생성 비동기 요청이 모두 마무리될 때까지 모듈 destroy 정지

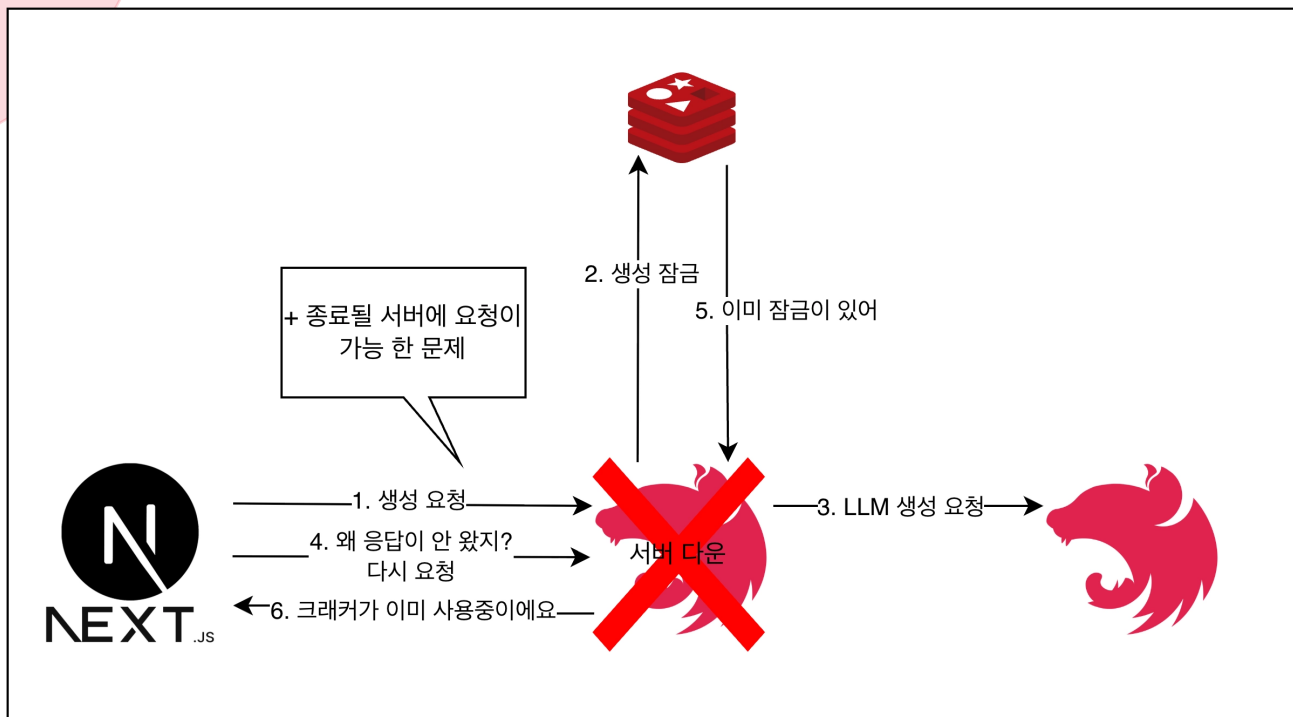
LLM 생성

```
/** 서버 종료 시그널을 보냅니다. */
async onModuleDestroy(): Promise<void> {
  await this.namespace.server.close();

  for (const socket of this.namespace.sockets.values()) {
    socket.emit(DEFAULT_SOCKET_EMIT_EVENT_NAME.DISCONNECT_SERVER,
SocketResponseDto.of(undefined));
    socket.disconnect();
  }

  // 비동기 생성이 모두 끝날 때까지 딜레이
  // 생성 시작 타임아웃 + 생성 타임아웃 시간에 5초 버퍼를 추가합니다.
  if (this.configService.get("NODE_ENV") === "prod") {
    await delay(this.STREAM_DEFAULT_TIMEOUT + this.STREAM_REQUEST_TIMEOUT + 5000);
  }
}
```


08 해결 방안 - nestjs 라이프 사이클 조정



08 해결 방안 - nestjs 라이프 사이클 조정

nes

When the application receives a termination signal it will call any registered `onModuleDestroy()` , `beforeApplicationShutdown()` , then `onApplicationShutdown()` methods (in the sequence described above) with the corresponding signal as the first parameter. **If a registered function awaits an asynchronous call (returns a promise), Nest will not continue in the sequence until the promise is resolved or rejected.**

08 해결 방안 - nestjs 라이프 사이클 조정

중앙 S

```
// Spring의 @Service와 비슷
@Injectable()
export class ShutdownService implements OnModuleDestroy {
  private readonly cleanupTasks: Array<() => Promise<void>> = [];

  // Spring의 @PreDestroy 혹은 DisposableBean.destroy와 비슷
  async onModuleDestroy() {
    await Promise.allSettled(this.cleanupTasks.map((task) => task()));
  }

  /** onModuleDestroy로 동시에 수행해야 할 작업을 추가 */
  addCleanupTask(task: () => Promise<void>) {
    this.cleanupTasks.push(task);
  }
}
```

08 해결 방안 - nestjs 라이프 사이클 조정

중앙 S

```
export class ChatV3Gateway implements OnModuleInit, OnGatewayDisconnect, OnGatewayInit {  
  ...  
  
  constructor(  
    ...  
    private readonly shutdownService: ShutdownService  
  ) {}  
  
  // Spring의 @PostConstruct, InitializingBean.afterPropertiesSet와 동일  
  // 모듈 생성 시 수행해야 할 종료 콜백을 등록  
  async onModuleInit(): Promise<void> {  
    this.shutdownService.addCleanupTask(this.destroy.bind(this));  
  }  
}
```

08 해결 방안 - nestjs 라이프 사이클 조정



Russell VIP

2025년 12월 5일 15:08

테리맨이 배포 할때마다 뜨던 크래커 사용중입니다 예러 고쳤어요
(이제 테리를 소켓 마스터로 부를려고 합니다)

대단하다 테리맨!



1개의 댓글



Jake VIP

2025년 12월 5일 16:36

테리 믿고 있었습니다

SECTION 09

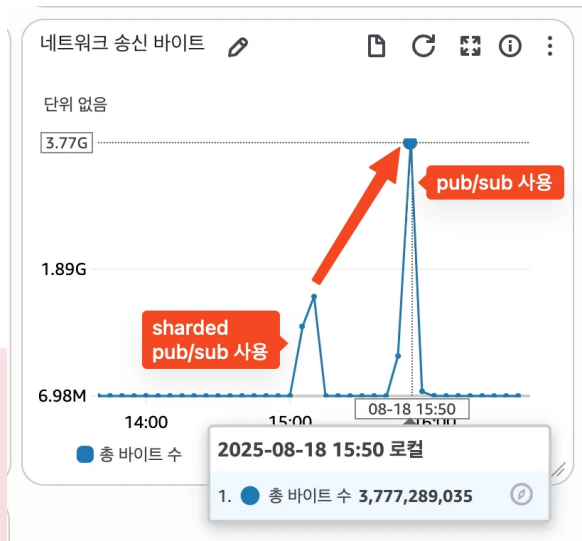
아쉬운 점

09

09

아쉬운 점

- 네트워크 트래픽을 획기적으로 더 경감시키지 못함
- pub/sub은 클러스터 버스를 통해 모든 노드에 브로드캐스팅 → 네트워크 송신 트래픽 과다
- Sharded pub/sub 도입 시 약 60% 송신 트래픽 절감 가능 (부하 테스트 결과 확인)
- 그러나 ElastiCache 클러스터 브로커용 라이브러리 미지원으로 도입 보류



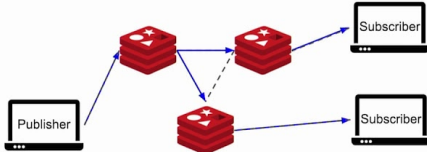
09

아쉬운 점



redisday
TLV 2018

Redis Cluster - Architecture



```
graph LR; P[Publisher] --> N1[Node 1]; P --> N2[Node 2]; P --> N3[Node 3]; N1 <--> N2; N2 <--> N3; N3 <--> N1; N1 --> S1[Subscriber]; N2 --> S1; N3 --> S2[Subscriber];
```

The diagram illustrates the Redis Cluster architecture. A Publisher (laptop icon) sends data to three Redis nodes (stacked red cubes). The nodes are interconnected in a mesh topology: Node 1 is connected to Node 2 and Node 3; Node 2 is connected to Node 1 and Node 3; Node 3 is connected to Node 1 and Node 2. Node 1 and Node 2 both send data to a Subscriber (laptop icon), while Node 3 sends data to another Subscriber (laptop icon).

감사합니다.
